



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

**Отчет по дисциплине «Проектная деятельность»
по домашней работе на семинаре №1**

Тема: «Простой детектор без ML»

**Проверил
к.т.н. доцент**

Ковалев И.А.

подпись

**Выполнила
студентка группы ИДБ-25-06**

Якунина В.С.

подпись

Оглавление

ВВЕДЕНИЕ.....3

ПРОГРАММА.....3

ЗАКЛЮЧЕНИЕ.....6

ВВЕДЕНИЕ

Целью работы является знакомство с логикой анализа телеметрии космического аппарата и понимание ограничения правил, заданных человеком, до перехода к машинному обучению. Ссылка на [github](#) с программой.

ПРОГРАММА

Для выполнения задачи необходимо было разработать алгоритм, который анализирует данные о характеристиках спутника, анализирует и обрабатывает их.

```
import pandas as pd
import matplotlib.pyplot as plt

def get_intervals(df, col='anomaly'):
    mask = df[col] == 1
    grp = (mask != mask.shift()).cumsum()

    intervals = (
        df[mask]
        .groupby(grp)
        .agg(
            start=('timestamp', 'first'),
            end=('timestamp', 'last'),
            rows=('timestamp', 'size')
        )
        .reset_index(drop=True)
    )
    intervals['duration_min'] = (
        (intervals['end'] - intervals['start']).dt.total_seconds() / 60
    )
    return intervals

def detect_sensitive(row):
    return int(
        row['temperature'] > 55 or
        row['voltage'] < 24 or
```

```

    row['current'] > 6 or
    row['angular_velocity'] > 0.5
)

def detect_conservative(row):
    return int(
        row['temperature'] > 65 or
        row['voltage'] < 23 or
        row['current'] > 9 or
        row['angular_velocity'] > 1.5
    )

def main():
    filename = "telemetry_dzz_sem1.csv"

    df = pd.read_csv(filename, parse_dates=['timestamp'])
    df['timestamp'] = pd.to_datetime(df['timestamp'])
    df = df.sort_values('timestamp').reset_index(drop=True)

    print('TELEMETRY (5)')
    print(df.head())
    print()
    print("Sample size:", len(df))
    print(df.describe())
    print()

    df['anomaly_sens'] = df.apply(detect_sensitive, axis=1)

    df['anomaly_cons'] = df.apply(detect_conservative, axis=1)

    print('WITH ANOMALY (5)')
    print(df.head())
    print()

    intervals_sens = get_intervals(df, 'anomaly_sens')
    intervals_cons = get_intervals(df, 'anomaly_cons')

    print("INTERVALS - sensitive detector")
    print(intervals_sens)
    print()
    print("INTERVALS - conservative detector")
    print(intervals_cons)
    print()

    print("DETECTOR COMPARISON")
    print(f"Sensitive:    anomaly rows = {df['anomaly_sens'].sum()}, "
          f"intervals = {len(intervals_sens)}")

```

```

print(f"Conservative: anomaly rows = {df['anomaly_cons'].sum()}, "
      f"intervals = {len(intervals_cons)}")
print()

params = ['temperature', 'voltage', 'current', 'angular_velocity']

for col in params:
    plt.figure(figsize=(12, 4))
    plt.plot(df['timestamp'], df[col], label=col, color='tab:blue')

    plt.scatter(
        df.loc[df['anomaly_sens'] == 1, 'timestamp'],
        df.loc[df['anomaly_sens'] == 1, col],
        color='orange', s=25, label='sensitive', zorder=5
    )
    plt.scatter(
        df.loc[df['anomaly_cons'] == 1, 'timestamp'],
        df.loc[df['anomaly_cons'] == 1, col],
        color='red', s=30, marker='x', label='conservative', zorder=6
    )

    for _, row in intervals_sens.iterrows():
        plt.axvspan(row['start'], row['end'], color='orange', alpha=0.10)
    for _, row in intervals_cons.iterrows():
        plt.axvspan(row['start'], row['end'], color='red', alpha=0.15)

    plt.title(f'Telemetry: {col}')
    plt.xlabel('timestamp')
    plt.ylabel(col)

print()
print('CLOSE FIGURES')
print()

plt.show()

if __name__ == "__main__":
    main()

```

ЗАКЛЮЧЕНИЕ

При выполнении задания был разработан простой детектор без машинного обучения. Ссылка на [github](#) с программой.